CSAI 5000: Fundamentals of Cybersecurity

Lab 7: Operating System security hands-on

Duration: 90 minutes
Prerequisites: Lecture 7 (Operating System security)
Environment: Linux VM (Kali or similar recommended)

Lab objectives

By completing this lab, you will:

- Configure and analyze Linux file permissions
- Understand SUID/SGID security implications
- Harden SSH and system configurations
- Use security auditing tools (Lynis)
- Analyze authentication and system logs
- Identify common privilege escalation vectors

Setup (5 minutes)

Verify your environment

You need a Linux system with root/sudo access. Verify:

```
# Check your user
whoami
id

# Verify sudo access
sudo whoami
```

Create lab directory and test users

```
# Create lab directory
mkdir -p ~/lab5
cd ~/lab5

# Create test users for exercises
sudo useradd -m -s /bin/bash alice
sudo useradd -m -s /bin/bash bob
sudo useradd -m -s /bin/bash charlie

# Set passwords
echo "alice:password123" | sudo chpasswd
echo "bob:password456" | sudo chpasswd
echo "charlie:password789" | sudo chpasswd

# Create a test group
sudo groupadd developers
sudo usermod -aG developers alice
sudo usermod -aG developers bob
```

Part 1: File permissions (20 minutes)

Exercise 1.1: Basic permissions

Step 1: Create test files and directories:

```
sudo mkdir -p ~/lab5/permissions_test
cd ~/lab5/permissions_test

sudo touch public.txt
sudo touch private.txt
sudo touch script.sh
sudo mkdir shared_folder
```

Step 2: View current permissions:

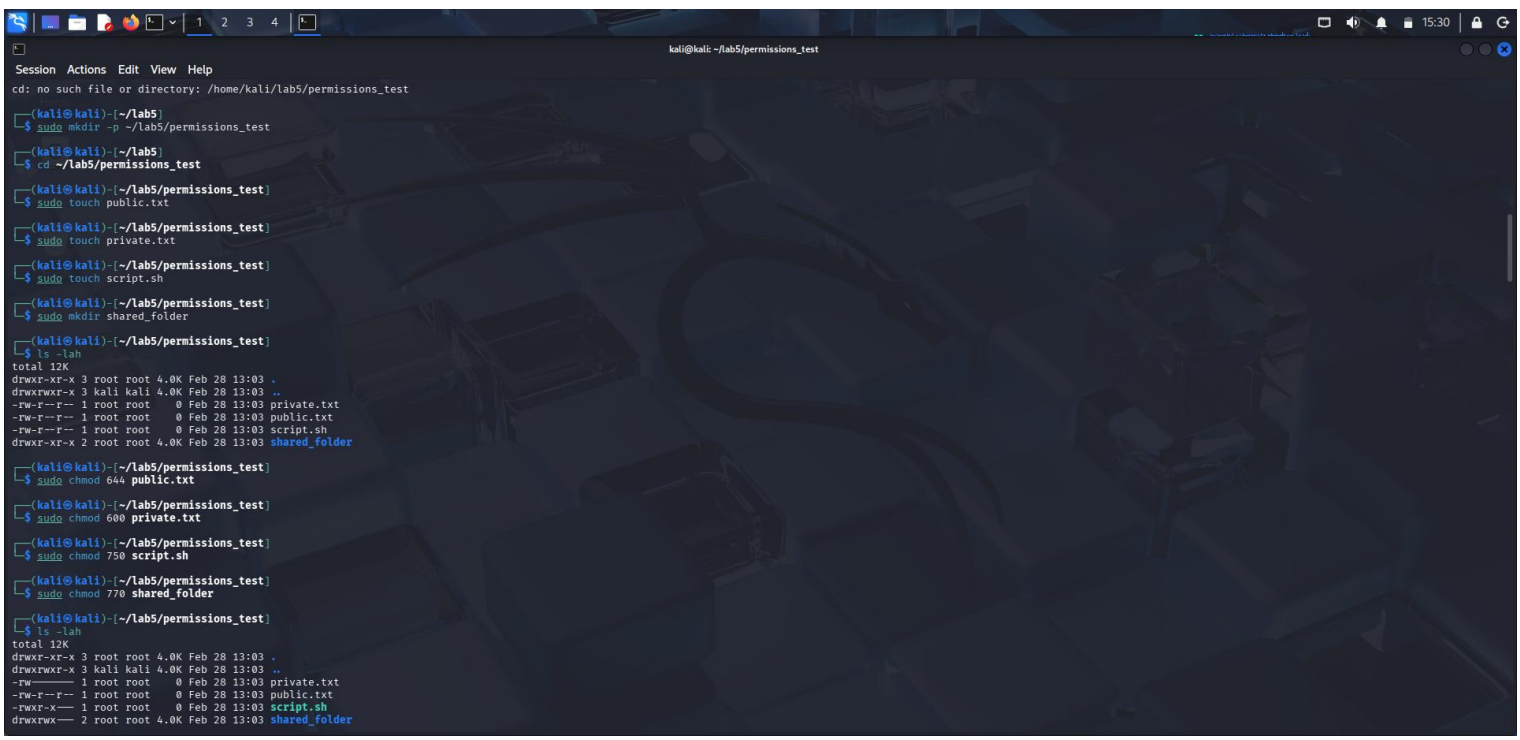
```
ls -lah
```

Step 3: Set specific permissions:

```
# Public file: everyone can read
sudo chmod 644 public.txt

# Private file: only owner can read/write
sudo chmod 600 private.txt

# Script: owner can do all, group can read/execute, others nothing
sudo chmod 750 script.sh

# Shared folder: owner full, group read/write, others nothing
sudo chmod 770 shared_folder
```

Step 4: Verify permissions:

```
ls -lah
```



Question 1: Translate the following permissions to numeric (octal) format:

- -rwxr-xr-- = 750
- -rw-rw---- = 660
- -r-------- = 400

Exercise 1.2: Ownership and groups

Step 1: Change ownership:

```
# Give public.txt to alice
sudo chown alice:developers public.txt

# Give private.txt to bob
sudo chown bob:bob private.txt

# Give shared_folder to alice with developers group
sudo chown alice:developers shared_folder
```

Step 2: Test access as different users:

```
# Test as alice
sudo -u alice cat public.txt
sudo -u alice cat private.txt

# Test as bob
sudo -u bob cat public.txt
sudo -u bob cat private.txt
```

Question 2: Can alice read private.txt? Can bob? Explain why based on the permissions and ownership.

Answer: Alice can not read private.txt file because we didn't give the ownership access to private.txt to alice as the file is accessible to owners only. Bob can access the private.txt file because using the chown (Change Ownner) command we changed the ownership for bob access private.txt file.

Exercise 1.3: umask

Step 1: Check current umask:

```
umask
```

Step 2: Create files with default umask:

```
touch /tmp/default_file
mkdir /tmp/default_dir
ls -la /tmp/default_file /tmp/default_dir
```

Step 3: Change umask and create new files:

```
umask 077
touch /tmp/secure_file
mkdir /tmp/secure_dir
ls -lah /tmp/secure_file /tmp/secure_dir
```

Question 3: With umask 077, what permissions do new files and directories get? Why is this more secure?

Answer: The default umask on most Linux systems is 022, which means new files get 644 and directories get 755 giving every user on the system read access to your files by default. Whereas, umask 077 enforces least privilege from the moment a file is created. No one except the owner can read, write, or access anything

Exercise 1.4: Access Control Lists (ACLs)

Step 1: Install ACL tools if needed:

```
sudo apt install acl -y
```

Step 2: Create a file and add ACL:

```
cd ~/lab5/permissions_test
sudo touch acl_test.txt
sudo chown alice:alice acl_test.txt
sudo chmod 600 acl_test.txt

# Grant charlie read access without changing ownership
sudo setfacl -m u:charlie:r acl_test.txt
```

Step 3: View and test ACL:

```
getfacl acl_test.txt
```

```
# Test access
sudo -u charlie cat acl_test.txt
sudo -u bob cat acl_test.txt
```

Question 4: How do ACLs provide more flexibility than standard Unix permissions?

Answer: Standard Unix permissions only allow three permissions sets owner, group and others. Whereas ACLs (Access Control Lists) allow you to assign permissions to any number of specific users or groups on a single file or directory, independently of the owner/group/others model.

Part 2: SUID/SGID analysis (15 minutes)

Exercise 2.1: Find SUID binaries

Step 1: Find all SUID files on the system:

```
sudo find / -perm -4000 -type f 2>/dev/null | head -20
```

Step 2: Examine a common SUID binary:

```
ls -la /usr/bin/passwd
ls -la /usr/bin/sudo
```

Question 5: Why does `/usr/bin/passwd` need the SUID bit? What would happen if it didn't have it?

Answer: /usr/bin/passwd needs the SUID bit because changing a password requires writing to /etc/shadow, which is only writable by root. When a regular user runs passwd, the SUID bit makes the process execute as root temporarily, giving it the access needed to update the password file. Without SUID, the process would fail with a permission denied error and users would be completely unable to change their own passwords.

Exercise 2.2: SUID security risks

Step 1: Create a vulnerable SUID program (for demonstration):

```
cd ~/lab5

# Create a simple C program
sudo tee suid_demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>

int main(int argc, char *argv[]) {
    printf("Real UID: %d\n", getuid());
    printf("Effective UID: %d\n", geteuid());

    if (argc > 1) {
        // Dangerous: executes user-provided command
        system(argv[1]);
    }
    return 0;
}
EOF

# Compile and set SUID
sudo gcc -o suid_demo suid_demo.c
sudo chown root:root suid_demo
sudo chmod 4755 suid_demo
```

Step 2: Test the SUID program:

```
./suid_demo "whoami"
./suid_demo "id"
```

Question 6: What makes this SUID program dangerous? How could an attacker abuse it?

Answer: The program passes user input directly into system() while running as root via the SUID bit. system() executes whatever string it receives as a shell command with no validation, or restriction on what that command can be. System() invokes /bin/sh -c internally, meaning the attacker can chain commands using shell operators.

Step 3: Clean up the vulnerable program:

```
sudo rm ~/lab5/suid_demo ~/lab5/suid_demo.c
```

Exercise 2.3: Find SGID binaries and directories

```
# Find SGID files
sudo find / -perm -2000 -type f 2>/dev/null | head -10

# Find SGID directories
sudo find / -perm -2000 -type d 2>/dev/null | head -10
```

Question 7: What is the purpose of SGID on a directory? Give a practical example.

Answer: When the SGID (Set Group ID) bit is set on a directory, any new files or subdirectories created inside automatically inherit the group ownership of the directory rather than the primary group of the user who created them.

Example: A development team has three members Alice, Bob, and Charlie all in the developers group. They share a project directory: Without SGID, every file would belong to the creator's personal group and teammates would need explicit permission changes to collaborate.

Part 3: SSH hardening (15 minutes)

Exercise 3.1: Examine current SSH configuration

Step 1: View the current SSH configuration:

```
sudo cat /etc/ssh/sshd_config | grep -v "^#" | grep -v "^$"
```

Step 2: Check critical settings:

```
grep -E "^(PermitRootLogin|PasswordAuthentication|PubkeyAuthentication|MaxAuthTries|Protocol)" /etc/ssh/sshd_config
```

Question 8: What is the current value of `PermitRootLogin`? What should it be for a secure configuration?

Answer: The default value on most Ubuntu systems is prohibit-password, meaning root can log in via SSH key but not via password. For a secure configuration it should be set to **no**, which completely disables root login over SSH. This forces admins to log in as a regular user and use sudo for privileged actions creating a clear audit trail and requiring an attacker to compromise two accounts instead of one.

Exercise 3.2: Create a hardened SSH configuration

Step 1: Backup the original configuration:

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup
```

Step 2: Create a hardening configuration file:

```
sudo tee /etc/ssh/sshd_config.d/hardening.conf << 'EOF'
# CSAI 5000 Lab 7 - SSH Hardening

# Disable root login
PermitRootLogin no

# Limit authentication attempts
MaxAuthTries 3

# Set login grace time
LoginGraceTime 60

# Disable empty passwords
PermitEmptyPasswords no

# Enable public key authentication
PubkeyAuthentication yes

# Disable X11 forwarding (if not needed)
X11Forwarding no

# Set client alive interval (disconnect idle sessions)
ClientAliveInterval 300
ClientAliveCountMax 2

# Only allow specific users (commented out for lab)
# AllowUsers alice bob

# Use only SSH protocol 2
Protocol 2

# Disable host-based authentication
```

```
HostbasedAuthentication no

# Ignore rhosts
IgnoreRhosts yes
EOF
```

Step 3: Validate the configuration:

```
sudo sshd -t
```

If no errors, the configuration is valid.

Note: In a production environment, you would restart SSH:

```
# DO NOT RUN IN LAB if you're connected via SSH
sudo systemctl restart sshd
```

Question 9: Why is disabling `PermitRootLogin` important? What is the risk of allowing root SSH access?

Answer: Disabling PermitRootLogin is important because it removes direct root access over SSH, forcing anyone who needs privileges to first log in as a regular user and then use sudo. This means every privileged action is logged and attributable to a specific person.

The risks of allowing root SSH access are that if an attacker successfully brute-forces, steals, or guesses the root credentials they have immediate and unrestricted control over the entire system with no additional barrier to overcome

Exercise 3.3: Generate SSH key pair

Step 1: Generate an SSH key pair (if you don't have one):

```
ssh-keygen -t ed25519 -C "lab5-key" -f ~/.ssh/lab5_key
```

Step 2: View the public key:

```
cat ~/.ssh/lab5_key.pub
```

Step 3: Understand authorized_keys:

```
ls -la ~/.ssh/
cat ~/.ssh/authorized_keys 2>/dev/null || echo "No authorized_keys file yet"
```

Question 10: Why is key-based authentication more secure than password authentication?

Answer: Key-based authentication is more secure than password authentication because a private key is cryptographically far stronger than any password and is never transmitted over the network, eliminating the risk of interception. Unlike passwords, keys cannot be brute-forced, phished, or guessed an attacker who does not possess the private key file has no attack vector.

Part 4: Security auditing with Lynis (20 minutes)

Exercise 4.1: Install and run Lynis

Step 1: Install Lynis:

```
sudo apt update
sudo apt install lynis -y
```

Step 2: Run a system audit:

```
sudo lynis audit system
```

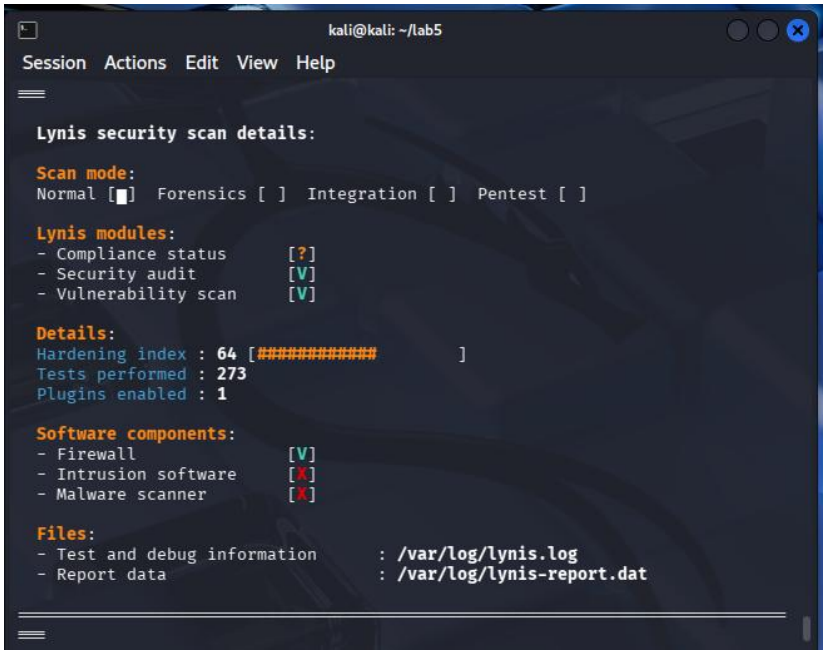
This will take a few minutes. Watch the output.

Step 3: Review the results:

```
# View the report
sudo cat /var/log/lynis-report.dat | head -100

# Check the hardening index
grep "hardening_index" /var/log/lynis-report.dat
```

Question 11: What is your system's hardening index score? What does this score indicate?

Answer: The hardening index score of the system is 64 which indicates that a system has some vulnerabilities It means your system has basic security controls in place but has significant room for improvement. It is above the poor range but falls short of what would be considered a well-hardened system in a production or security-sensitive environment.



Exercise 4.2: Analyze Lynis warnings

Step 1: Find warnings and suggestions:

```
sudo grep -E "^warning|^suggestion" /var/log/lynis-report.dat | head -20
```

Step 2: Look at specific test results:

```
sudo lynis show details FIRE-4512 2>/dev/null || echo "Test ID may vary"
```

Question 12: List 3 warnings or suggestions from Lynis and explain what security risk each addresses.

Answer:

1. ACCT-9628 — auditd not enabled: Without it, there is no forensic trail of what happened on the system. If an attacker gains access, you have no way to determine what they did, what files they accessed, or how they got in.

2. AUTH-9328 — Default umask too permissive: This addresses the risk of sensitive files being readable by all users on the system config files, scripts, or data files with credentials could be exposed to any logged-in user.

3. DEB-0880 — fail2ban not installed: Without fail2ban, your SSH service is open to unlimited brute force attempts. An attacker can try thousands of passwords per second with no automatic blocking.

Exercise 4.3: Targeted audits

Step 1: Run specific test categories:

```
# Test only authentication
sudo lynis audit system --tests-from-group authentication

# Test only file permissions
sudo lynis audit system --tests-from-group file_permissions
```

Part 5: Log analysis (15 minutes)

Exercise 5.1: Authentication logs

Step 1: View recent authentication events:

```
# Kali
sudo tail -50 /var/log/auth.log

# Or using journalctl
sudo journalctl -u ssh --since "1 hour ago"
```

Step 2: Find failed login attempts:

```
sudo grep "Failed password" /var/log/auth.log | tail -10
```

Step 3: Find successful logins:

```
sudo grep "Accepted" /var/log/auth.log | tail -10
```

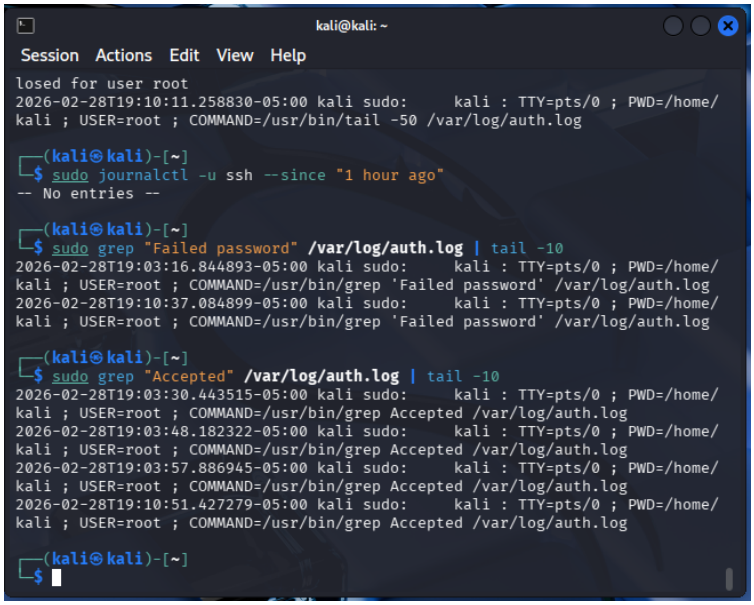
Question 13: What information is logged for a failed SSH login attempt? Why is this useful for security?

Answer:

Detecting brute force attacks: multiple failed attempts from the same IP in a short time is a clear indicator of automated password guessing. Tools like fail2ban read these logs to automatically block offending IPs.

Threat intelligence: the source IP can be cross-referenced against threat intelligence databases to identify known malicious hosts or botnets.

Incident respons: if a system is later compromised, failed login logs help reconstruct the timeline and determine whether the attacker succeeded after multiple failures.



Exercise 5.2: Generate and analyze log entries

Step 1: Generate some failed login attempts:

```
# Try to su to a user with wrong password (will fail)
su - alice << 'EOF'
wrongpassword
EOF
```

Step 2: Check the logs:

```
sudo tail -5 /var/log/auth.log
```

Exercise 5.3: System logs

Step 1: View kernel messages:

```
sudo dmesg | tail -20
```

Step 2: View system logs:

```
sudo journalctl -p err --since "24 hours ago" | head -30
```

Step 3: View service-specific logs:

```
sudo journalctl -u cron --since "1 hour ago"
```

Question 14: Why is it important to forward logs to a central logging server (SIEM)?

Answer: Forwarding logs to a central SIEM is important because if an attacker compromises a system they can delete local logs to cover their tracks, but logs already forwarded to a remote server cannot be tampered with. A SIEM also correlates events across multiple systems simultaneously, allowing security teams to detect patterns like brute force campaigns that would be invisible when looking at individual machines.

Part 6: Privilege escalation vectors (10 minutes)

Exercise 6.1: Check for common misconfigurations

Step 1: Check sudo configuration:

```
sudo cat /etc/sudoers
sudo ls -la /etc/sudoers.d/
```

Step 2: Find world-writable files:

```
sudo find /etc -perm -002 -type f 2>/dev/null
```

Step 3: Check for writable PATH directories:

```
echo $PATH | tr ':' '\n' | while read dir; do
    if [ -w "$dir" ]; then
        echo "WRITABLE: $dir"
    fi
done
```

Step 4: Find files with no owner:

```
sudo find / -nouser -o -nogroup 2>/dev/null | head -10
```

Question 15: If a user can write to a directory in root's PATH, how could they escalate privileges?

Answer: If a user can write to a directory in root's PATH, they can create a malicious script with the same name as a command root regularly runs for example naming it service or python. When root executes that command without specifying the full path, the system searches PATH directories in order and finds the malicious version first, executing it with root privileges. The attacker's script can then do anything.

Exercise 6.2: Check cron jobs

Step 1: View system cron jobs:

```
cat /etc/crontab
ls -lah /etc/cron.d/
ls -lah /etc/cron.daily/
```

Step 2: Check for user cron jobs:

```
sudo ls -la /var/spool/cron/crontabs/ 2>/dev/null
```

Question 16: Why are cron jobs running as root a potential privilege escalation risk?

Answer: Cron jobs running as root are a privilege escalation risk because they execute automatically with full root privileges, and if an attacker can modify the script or binary that the cron job calls, every execution of that job runs their malicious code as root with no user interaction required.

Cleanup

```
# Remove test users
sudo userdel -r alice 2>/dev/null
sudo userdel -r bob 2>/dev/null
sudo userdel -r charlie 2>/dev/null
sudo groupdel developers 2>/dev/null

# Remove lab directory
sudo rm -rf ~/lab5

# Remove temp files
rm -f /tmp/default_file /tmp/secure_file
rm -rf /tmp/default_dir /tmp/secure_dir

# Restore SSH config if modified
sudo rm /etc/ssh/sshd_config.d/hardening.conf 2>/dev/null

# Remove SSH keys created for lab
rm -f ~/.ssh/lab5_key ~/.ssh/lab5_key.pub
```

Lab submission

Submit the following:

1. Completed answer sheet with all 16 questions answered
2. Screenshot: Output of `ls -lah` showing permission changes (Exercise 1.1)
3. Screenshot: Lynis hardening index and summary (Exercise 4.1)
4. Screenshot: Failed login attempts in auth.log (Exercise 5.1)

Due: Before next lab session

Challenge exercises (optional)

For students who finish early:

1. **Create a hardening script:** Write a bash script that implements at least 10 hardening measures from the CIS Benchmark.

2. **Build a privilege escalation lab:** Set up an intentionally vulnerable user with sudo misconfigurations and document the escalation path.

3. **Implement file integrity monitoring:** Install and configure AIDE to monitor critical system files:

```
sudo apt install aide
sudo aideinit
sudo aide --check
```

7. **SELinux/AppArmor Exploration:** If using a system with SELinux or AppArmor, create a custom policy for a simple application.

Resources

Documentation:

- CIS Benchmarks: https://www.cisecurity.org/cis-benchmarks/
- Linux Security Modules: https://www.kernel.org/doc/html/latest/admin-guide/LSM/
- Lynis Documentation: https://cisofy.com/documentation/lynis/

Practice:

- OverTheWire Bandit: https://overthewire.org/wargames/bandit/
- HackTheBox Linux Machines
- TryHackMe Linux Privilege Escalation

Next Lecture: Incident response & forensics